whereexpr

A fast, expressive rule-based filtering engine for Rust

Evaluate boolean expressions over any data structure

Gavrilut Dragos

v 0.3.1

Agenda

01

What is whereexpr – Core Concepts

Conditions, predicates and operations, expressions, three-valued logic

02

Example and Rust implementation

The Student example, Attributes written by hand, matches vs evaluate

03

Derive

#[derive(Attributes)], field types and options, properties, enums

04

Rules

Rule text, from_rule_str, RuleParser and type dispatch, metadata (with meta)

05

Benchmarks

Speed and memory of the search operations

SECTION 1 OF 5

01

What is whereexpr

- Core Concepts -

Conditions, predicates and operations, expressions, three-valued logic

What is whereexpr?

▶ Stream Filtering

Drop or keep events on a live stream based on boolean rules

Detection Rules

Write custom detections that trigger when conditions match

🛡 Exclusions

Suppress known-good events from an existing detection

🔍 Data Querying

Filter rows from a database like a WHERE clause

Core Concepts

Attribute

A read-only data specific to an object (data-member or method)

Implementation

struct **Student**

- Fields
  - *Name*
  - *MathGrade*
  - *EnglishGrade*
  - *Age*
- Methods
  - *AverageGrade*
  - *FamilyName*



Attributes associated with Student

Name	*String*
MathGrade	*Integer*
EnglishGrade	*Integer*
Age	*Integer*
AverageGrade	*Integer*
Family Name	*String*

Supported Data Types

- **Numeric** (u8, u16, u32, u64, i8, i16, i32, i64, f32, f64)
- **Boolean** (bool)
- **Strings** (&str)
- **Path** (&str)
- **Ip addresses**
- **Hash-es** (128 bits, 160 bits, 256 bits) – e.g. MD5, SHA1, SHA256
- **DateTime** (seconds from 1970)

Core Concepts

Attribute

A read-only data specific to an object (data-member or method)

Predicate

A compiled operation over a constant value(s)

Supported operations

- **Comparisons** (> , < , different, equals, etc)
- **String/Buffer based** (starts with, contains, etc)
- **List** (is-one-of, etc)
- **Regular expressions** (matches …)
- **Ranges** (between X and Y)

* ***Each operation also supports the NOT operator***

Constant Value

- **Scalar**
- **Array**

* *A constant value supports the same type as an attribute*

Examples

> 10	*comparison*
starts-with "xyz"	*string based*
is-one-of ["John", "Alex", "Gabriel", "Martin"]	*list based*
in-range [192.168.0.1 , 192.168.0.255]	*range operation*
glob-match ["*.exe", "*.dll", "*.p??"]	*match*

Core Concepts

Attribute

A read-only data specific to an object (data-member or method)

Predicate

A compiled operation over a constant value(s)

Operation Aliases

- **Equal** (is, equals, eq, ==)
- **Different** (is-not, not-equals, neq, !=)
- **GreaterThan** (gt, >, greater-than)
- …

** Underline (_) and hyphen (-) are ignored when parsing an operation name. This means that* ***is-not*** , ***is_not*** , ***isnot*** *are the same.*

Operation Classes

- **Equality** (is, is-not)
- **Comparison** (> , < , >= , <=)
- **BufferFind** (starts-with, contains, ends-with)
- **Range**
- **Lists** (is-one-of, is-not-one-of)
- **AutomatFind** (starts-with-one-of, contains-one-of, ends-with-one-of)
- **RegExp** (re-match, glob-match)
- **AutomatRegExp** (glob-match with a list)

Core Concepts

Attribute

A read-only data specific to an object (data-member or method)

Predicate

A compiled operation over a constant value(s)

Data Type / Operation class associations

	EQ	CMP	BUF	R	LST	AF	RE	ARE
Numeric	√	√		√	√			
Bool	√							
Enum	√				√			
String	√		√		√	√	√	
Path	√		√		√	√	√	√
IP	√			√	√			
Hashes	√				√			
DateTime	√	√		√				

Legend

EQ – equality, **CMP** – comparisons, **BUF** – buffer find, **R** – range,
LST – list, **AF** – automat find, **RE** – regular expression, **ARE** – automat re

Core Concepts

Attribute

A read-only data specific to an object (data-member or method)

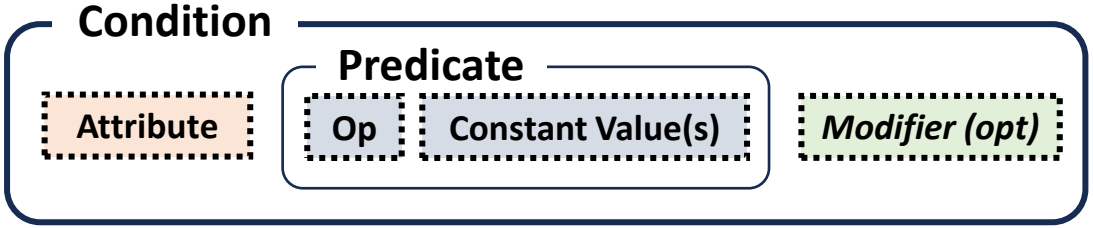
Predicate

A compiled operation over a constant value(s)

Condition

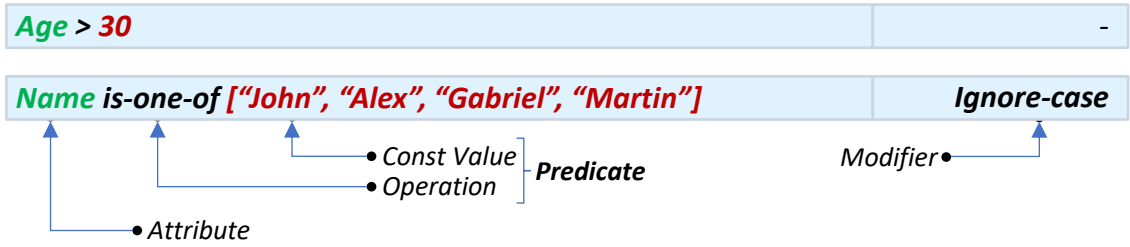
A tuple formed out of an attribute and a predicate

Architecture



- **Modifier** – optional component that changes how the predicate is evaluated (e.g. if ***ignore-case*** should be used or not)

Examples



Core Concepts

Attribute

A read-only data specific to an object (data-member or method)

Predicate

A compiled operation over a constant value(s)

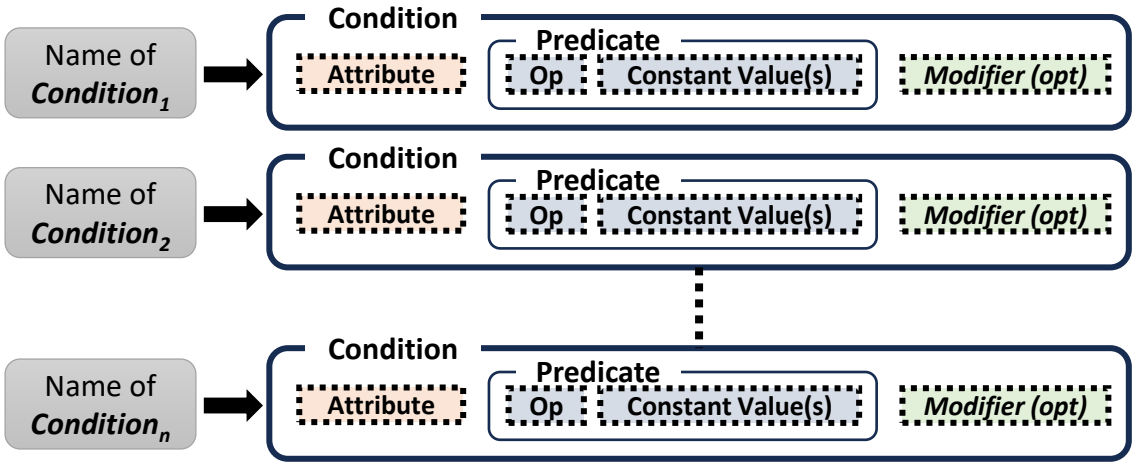
Condition

A tuple formed out of an attribute and a predicate

Expression

A compiled logical evaluation over a set of conditions

Architecture (set of conditions)



* *Each name is unique (they act as a* ***key*** *in a dictionary/map)*

Architecture (logical evaluation)

A logical expression using condition names and operator AND, OR, NOT as well as parentheses (…) e.g. $Cond_1$ AND $Cond_2$ AND ($Cond_3$ OR $Cond_4$)

Core Concepts

Condition

A tuple formed out of an attribute and a predicate

Expression

A compiled logical evaluation over a set of conditions

Unknown

A condition whose attribute is missing (get → None)

Kleene logic

How AND, OR and NOT combine true, false and unknown

Three-valued logic (truth tables)

AND	T	F	U
T	T	F	U
F	F	F	F
U	U	F	U

OR	T	F	U
T	T	T	T
F	T	F	U
U	T	U	U

a	NOT a
T	F
F	T
U	U

Rows: a · Columns: b · **T** = true **F** = false **U** = unknown

Rules

AND any false → false; else any unknown → unknown; else true
OR any true → true; else any unknown → unknown; else false
NOT swaps true and false; unknown stays unknown

Kleene logic (examples)

Examples (a, b, c, d are conditions)

Expression	a	b	c	d	Result	Why
`a AND b AND c AND d`	F	F	U	F	false	a false operand decides the AND
`a AND b AND c AND d`	F	F	U	T	false	a false operand decides the AND
`a AND b AND c AND d`	T	T	U	T	unknown	only c could still change the result
`a AND b AND c AND d`	T	T	T	T	true	all operands are true
`a OR b OR c OR d`	F	F	U	F	unknown	only c could still change the result
`a OR b OR c OR d`	F	F	U	T	true	a true operand decides the OR
`a AND b`	U	F			false	order does not matter ...
`a AND b`	F	U			false	... same result either way
`(a OR b) AND c`	U	T	T		true	a OR b is already true
`NOT (a AND b)`	U	T			unknown	NOT unknown is still unknown

Evaluation still short-circuits: AND stops at the first false and OR at the first true. An unknown operand does not stop it, because a later operand may still decide the result.

SECTION 2 OF 5

02

Example and Rust implementation

The Student example, Attributes written by hand, matches vs evaluate

Example (Student filtering)

Object / Structure

`struct` **Student**

- **Fields**
  - *Name*
  - *MathGrade*
  - *EnglishGrade*
  - *Age*
  - *Enroll*
  - *Birthday*
- **Methods**
  - *AverageGrade*
  - *FamilyName*

Attributes

Name	*String*
Grade.Math	*Integer*
Grade.English	*Integer*
Age	*Integer*
Grade.Average	*Integer*
Name.Family	*String*
Enroll.Year	*Integer*
Birthday.Month	*Integer*

Condition Set

Age-Is-Prime	*Age* ***is-one-of*** *[19,23,29,31]*
Common	*Name.Family* ***ends-with*** *"escu"* ignore-case
Old-Enroll	*Enroll.Year* ***in-range*** *[2010,2020]*
Passes-Courses	*Grade.Average* ***>=*** *5*
Born-in-dec	*BirthDay.Month* ***is*** *12*
Vowel-Name	*Name* ***starts-with-one-of*** *[A,E,I,O,U]*
Good-at-math	*Grade.Math* ***>=*** *8*

Expression

```
Age-Is-Prime OR Common OR (Old-Enroll AND Good-at-math)
OR NOT (Passes-Courses AND NOT (Born-in-dec OR Vowel-Name))
```

Example (Rust implementation)

1 **Structure Definition**

Code

```
struct Student {
    name: String,
    math_grade: u32,
    english_grade: u32,
    age: u32,
    enrolled: NaiveDate,
    birthday: NaiveDate,
}
```

Example (Rust implementation)

1. Structure Definition
2. Student implementation

Code

```
struct Student {…}
impl Student {
    const NAME: AttributeIndex           = AttributeIndex::new(0);
    const GRADE_MATH: AttributeIndex     = AttributeIndex::new(1);
    const GRADE_ENGLISH: AttributeIndex  = AttributeIndex::new(2);
    const GRADE_AVG: AttributeIndex      = AttributeIndex::new(3);
    const AGE: AttributeIndex            = AttributeIndex::new(4);
    const FAMILY_NAME: AttributeIndex    = AttributeIndex::new(5);
    const ENROLL_YEAR: AttributeIndex    = AttributeIndex::new(6);
    const BIRTHDAY_MONTH: AttributeIndex = AttributeIndex::new(7);

    fn family_name(&self) -> &str {
        self.name.split(' ').next().unwrap()
    }
    fn average_grade(&self) -> u32 {
        (self.math_grade + self.english_grade) / 2
    }
```

Example (Rust implementation)

1. Structure Definition
2. Student implementation
3. Attributes (type safety)

Code

```rust
impl Attributes for Student {
    const TYPE_ID: u64 = 0x12345678; // unique ID for Student type
    const TYPE_NAME: &'static str = "Student";
```

Example (Rust implementation)

1. Structure Definition
2. Student implementation
3. Attributes (type safety)
4. **Attributes (get values)**

Code

```rust
impl Attributes for Student {
    const TYPE_ID: u64 = 0x12345678; // unique ID for Student type
    const TYPE_NAME: &'static str = "Student";

    fn get(&self, idx: AttributeIndex) -> Option<Value<'_>> {
        match idx {
            Self::NAME => Some(Value::String(self.name.as_str())),
            Self::GRADE_MATH => Some(Value::U32(self.math_grade)),
            Self::GRADE_ENGLISH => Some(Value::U32(self.english_grade)),
            Self::GRADE_AVG => Some(Value::U32(self.average_grade())),
            Self::AGE => Some(Value::U32(self.age)),
            Self::FAMILY_NAME => Some(Value::String(self.family_name())),
            Self::ENROLL_YEAR => Some(Value::I32(self.enrolled.year())),
            Self::BIRTHDAY_MONTH => Some(Value::U32(self.birthday.month())),
            _ => None,
        }
    }
```

Example (Rust implementation)

1. Structure Definition
2. Student implementation
3. Attributes (type safety)
4. Attributes (get values)
5. **Attributes (type kind)**

Code

```
impl Attributes for Student {
    fn get(&self, idx: AttributeIndex) -> Option<Value<'_>> {…}

    fn kind(idx: AttributeIndex) -> Option<ValueKind> {
        match idx {
            Self::NAME => Some(ValueKind::String),
            Self::GRADE_MATH => Some(ValueKind::U32),
            Self::GRADE_ENGLISH => Some(ValueKind::U32),
            Self::GRADE_AVG => Some(ValueKind::U32),
            Self::AGE => Some(ValueKind::U32),
            Self::FAMILY_NAME => Some(ValueKind::String),
            Self::ENROLL_YEAR => Some(ValueKind::I32),
            Self::BIRTHDAY_MONTH => Some(ValueKind::U32),
            _ => None,
        }
    }
```

Example (Rust implementation)

1. Structure Definition
2. Student implementation
3. Attributes (type safety)
4. Attributes (get values)
5. Attributes (type kind)
6. **Attributes (indexes)**

Code

```rust
impl Attributes for Student {
    fn get(&self, idx: AttributeIndex) -> Option<Value<'_>> {…}
    fn kind(idx: AttributeIndex) -> Option<ValueKind> {…}
    // for compile time only
    fn index(name: &str) -> Option<AttributeIndex> {
        match name.to_ascii_lowercase().as_str() {
            "name" => Some(Self::NAME),
            "grade.math" => Some(Self::GRADE_MATH),
            "grade.english" => Some(Self::GRADE_ENGLISH),
            "grade.average" => Some(Self::GRADE_AVG),
            "age" => Some(Self::AGE),
            "name.family" => Some(Self::FAMILY_NAME),
            "enroll.year" => Some(Self::ENROLL_YEAR),
            "birthday.month" => Some(Self::BIRTHDAY_MONTH),
            _ => None,
        }
    }
```

Example (Rust implementation)

1. Structure Definition
2. Student implementation
3. Attributes (type safety)
4. Attributes (get values)
5. Attributes (type kind)
6. Attributes (indexes)
7. **Compile a whereexpr**

Code

```
let expr = ExpressionBuilder::<Student>::new()
    .add("age-is-prime", Condition::from_str("age is-one-of [19,23,29,31]"))
    .add("common", Condition::from_str(
                                    "name.family ends-with 'escu' {ignore-case}"))
    .add("old-enroll", Condition::from_str("enroll.year in-range [2010, 2020]"))
    .add("passes-courses", Condition::from_str("grade.average >= 5"))
    .add("born-in-dec", Condition::from_str("birthday.month is 12"))
    .add("vowel-name", Condition::from_str(
                        "name starts-with-one-of [A, E, I, O, U] {ignore-case}"))
    .add("good-at-math", Condition::from_str("grade.math >= 8"))
    .build("Age-Is-Prime OR Common OR (Old-Enroll AND Good-at-math) OR NOT
            (Passes-Courses AND NOT (Born-in-dec OR Vowel-Name))")
    .unwrap();
```

Example (Rust implementation)

1. Structure Definition
2. Student implementation
3. Attributes (type safety)
4. Attributes (get values)
5. Attributes (type kind)
6. Attributes (indexes)
7. Compile a whereexpr
8. **Create a Student Object**

Code

```
let expr = ExpressionBuilder::<Student>::new().add(…)…build(…).unwrap();

let student = Student {
    name: "Popescu Ion".to_string(),
    math_grade: 8,
    english_grade: 9,
    age: 20,
    enrolled: NaiveDate::from_ymd_opt(2020, 1, 1).unwrap(),
    birthday: NaiveDate::from_ymd_opt(1990, 12, 1).unwrap(),
};
```

Example (Rust implementation)

1. Structure Definition
2. Student implementation
3. Attributes (type safety)
4. Attributes (get values)
5. Attributes (type kind)
6. Attributes (indexes)
7. Compile a whereexpr
8. Create a Student Object
9. **Evaluate**

Code

```
let expr = ExpressionBuilder::<Student>::new().add(…)…build(…).unwrap();

let student = Student {…};

println!("matches: {}", expr.matches(&student) );

// three-valued result: Some(true), Some(false) or None (unknown)
match expr.evaluate(&student) {
    Some(true)  => println!("matches"),
    Some(false) => println!("does not match"),
    None        => println!("unknown (missing attribute)"),
}
```

Kleene logic (matches vs evaluate)

Two ways to evaluate

Result	matches	evaluate
true	`true`	`Some(true)`
false	`false`	`Some(false)`
unknown	**`false`**	**`None`**

`matches` treats unknown as no match, the way a SQL WHERE clause drops rows whose condition is NULL.

`evaluate` returns the full result, so you can tell "does not match" from "cannot tell".

Both panic if the object is not the type the expression was built for (with the `enable_type_check` feature).

Code

```
// Person { name: String, age: Option<u32> }
// get(): Self::AGE => self.age.map(Value::U32)

let expr = ExpressionBuilder::<Person>::new()
    .add("is-ion", Condition::from_str("name is Ion"))
    .add("adult",  Condition::from_str("age >= 18"))
    .build("is-ion AND adult").unwrap();

let ana = Person { name: "Ana".into(), age: None };
expr.matches(&ana);   // false
expr.evaluate(&ana);  // Some(false): the name decides

let ion = Person { name: "Ion".into(), age: None };
expr.matches(&ion);   // false
expr.evaluate(&ion);  // None: the missing age decides
```

SECTION 3 OF 5

03

Derive

#[derive(Attributes)], field types and options, properties, enums

Example (derive)

Code

```
#[derive(Attributes)]
#[whereexpr(name = "Student", id = 0x12345678)]
#[whereexpr(property = "fn family_name as name.family -> string")]
#[whereexpr(property = "fn average_grade as grade.average -> u32")]
#[whereexpr(property = "fn enroll_year as enroll.year -> i32")]
#[whereexpr(property = "fn birthday_month as birthday.month -> u32")]
struct Student {
    name: String,
    #[whereexpr(name = "grade.math")]
    math_grade: u32,
    #[whereexpr(name = "grade.english")]
    english_grade: u32,
    age: u32,
    #[whereexpr(ignore)]
    enrolled: NaiveDate,
    #[whereexpr(ignore)]
    birthday: NaiveDate,
}

// replaces the hand-written impl Attributes for Student
// (index constants, TYPE_ID, TYPE_NAME, get, kind, index)
```

What it means

`#[derive(Attributes)]`

Writes `impl Attributes` for you. Each kind comes from the field type: `String` is `string`, `u32` is `u32`.

`name = "Student", id = ...`

Struct options: TYPE_NAME and TYPE_ID, here the same values as the hand-written version.

`name = "grade.math"`

The name used in conditions. By default, the field name (age).

`ignore`

Not exposed. `NaiveDate` has no kind, so it must be ignored (or given `kind = ...`).

`property = "..."`

An attribute returned by a method, here for the derived values (next slide).

Example (derive)

Code

```
impl Student {
    fn family_name(&self) -> Option<&str> {
        self.name.split(' ').next()
    }
    fn average_grade(&self) -> Option<u32> {
        Some((self.math_grade + self.english_grade) / 2)
    }
    fn enroll_year(&self) -> Option<i32> {
        Some(self.enrolled.year())
    }
    fn birthday_month(&self) -> Option<u32> {
        Some(self.birthday.month())
    }
}

// the rest is unchanged: build, create, evaluate
let expr = ExpressionBuilder::<Student>::new()…build(…).unwrap();
println!("matches: {}", expr.matches(&student));
```

Generated for you

By hand	With derive
Index constants	generated
`TYPE_NAME`	`"Student"` (struct name)
`TYPE_ID`	FNV-64 hash of the name
`get`	fields + property methods
`kind`	inferred from field types
`index`	field names or `name = "…"`, exact case

Same `match` code as the hand-written version, so no runtime cost. Mistakes are compile errors.

Derive (field types)

How a field type becomes a kind

Field type	Kind	Note
`String, &str, Box<str>, Rc<str>, Cow<str>`	**`string`**	
`PathBuf, &Path, Cow<Path>`	**`path`**	a non UTF-8 path is unknown
`u8 … u64, i8 … i64, f32, f64`	**`the same`**	
`usize, isize`	**`u64, i64`**	a value that does not fit is unknown
`bool`	**`bool`**	
`IpAddr, Ipv4Addr, Ipv6Addr`	**`ip`**	
`SystemTime`	**`datetime`**	a time before 1970 is unknown
`[u8; 16], [u8; 20], [u8; 32]`	**`hash128…256`**	by length: 128, 160 or 256 bits
`Vec<u8>, &[u8]`	**`bytes`**	
`Vec<&str>, &[&str]`	**`string_list`**	
`Option<T>`	**`kind of T`**	None makes conditions unknown

Other types: NaiveDate, a type alias or your own type need `kind = ...` or `ignore`. An unsupported type is a compile error, not a surprise at run time.

Derive (options)

Options of #[whereexpr(...)]

Option	On	Effect	Default
`name = "grade.math"`	**field**	The name used in conditions: letters, digits, _ and `.`, starting with a letter	the field name
`kind = datetime`	**field**	The kind, when the type is ambiguous (u64 as a date) or unknown. Numbers can be widened, never narrowed	from the field type
`ignore`	**field**	Not exposed to conditions	exposed
`name = "Student"`	**struct**	TYPE_NAME, the type a rule names: `rule r: Student`	the struct name
`id = 0x12345678`	**struct**	TYPE_ID, checked by `enable_type_check`	FNV-64 hash of the name
`property = "fn … -> u32"`	**struct**	An attribute computed by a method (next slide)	none

Combine options in one attribute or repeat it: `#[whereexpr(name = "Student", id = 7)]`. `kind` values: `string, string_list, path, bytes, u8 … u64, i8 … i64, f32, f64, hash128, hash160, hash256, ip, datetime, bool`.

Properties

Code

```
#[derive(Attributes)]
#[whereexpr(property = "fn extension -> string")]
#[whereexpr(property = "fn word_count as words -> u32")]
struct Document {
    path: PathBuf,
    text: String,
}

impl Document {
    fn extension(&self) -> Option<&str> {
        self.path.extension()?.to_str()
    }
    fn word_count(&self) -> Option<u32> {
        Some(self.text.split_whitespace().count() as u32)
    }
}

// properties are used exactly like fields
rule long_md: Document {
    md   = "extension is md"
    long = "words > 100"
} where { md AND long }
```

Syntax

`fn <name> -> <kind>`

An attribute computed by `self.<name>()`, called only when a condition needs it.

`as <attribute>`

Optional name used in conditions; may contain `.` (`as meta.len`).

`fn path::func -> <kind>`

A path with `::` calls a free function: `path::func(self)`.

`Option<T>`

The return type, T matching the kind (`&str` for `string`). None makes the condition unknown.

Enums

Code

```
#[derive(WhereexprEnum)]
enum Color {
    #[whereexpr(aliases = ["red", "rosu"])]
    Red,      // id 0
    Green,    // id 1
    Blue,     // id 2
}

#[derive(Attributes)]
struct Car {
    #[whereexpr(kind = enum)]
    color: Color,
    #[whereexpr(kind = enum)]
    fuel: Option<Fuel>,
    doors: u8,
}

// names in conditions, numbers at run time
rule small_warm: Car {
    warm  = "color is-one-of [rosu, green]"
    small = "doors <= 3"
} where { warm AND small }
```

How to use

`#[derive(WhereexprEnum)]`

Variants get ids 0, 1, 2… in declaration order. Each is written as its name in lower case (`green`).

`aliases = ["red", "rosu"]`

The names to use instead of the default one. Names ignore case and must be unique.

`#[whereexpr(kind = enum)]`

On the field: `E`, `Option<E>` or `&E`. Never inferred from the type.

`is` `is-one-of`

With their negations (is-not, is-not-one-of), the only operations. An unknown name (`color is purple`) fails when the expression is built.

Enums (under the hood)

Code

```
// #[derive(WhereexprEnum)] writes:
impl WhereexprEnum for Color {
    fn as_u16(&self) -> u16 {
        match *self { Self::Red => 0, Self::Green => 1, Self::Blue => 2 }
    }
    fn as_u16_from_str(s: &str) -> Option<u16> {
        match fnv64_lowercase(s) {      // no names, only hashes
            0x89e9be1960f4c21c => Some(0),
            0xa3b3fc1ff114fa96 => Some(0),
            0x0f40f029637fecbc => Some(1),
            0xc5ccd29bc2dda64d => Some(2),
            _ => None,
        }
    }
}

// #[derive(Attributes)] adds, for the enum fields:
fn enum_value(idx: AttributeIndex, name: &str) -> Option<u16> {
    match idx.index() {
        0 => <Color as WhereexprEnum>::as_u16_from_str(name),
        _ => None,
    }
}
```

Names become numbers

When	What happens
Compile	Every name and alias is hashed. Two names with the same hash are a compile error.
Build	`enum_value` turns each name into an id when the expression is built. Unknown name: `UnknownEnumValue`.
`is`	Compares two u16 values.
`is-one-of`	One bit test: a u64 mask, or a bit set for ids from 64 up.

Only hashes are in the program, so the names cannot be read from it. Rules keep names, not ids: reordering the enum does not change what a rule means.

SECTION 4 OF 5

04 Rules

Rule text, from_rule_str, RuleParser and type dispatch, metadata (with meta)

Rules: syntax

Code

```
rule top-students: Student {
    adult = "age >= 18"
    good  = 'grade.average >= 9'
    math  = """grade.math >= 9"""
} where {
    adult AND (good OR math)
} with meta {
    owner = teachers
    tags  = [review, "end of year"]
}

rule adults: Student { adult = "age >= 18" } where { adult }
```

Syntax

`rule <name>: <Type> { … }`

<Type> is the type the rule is for (TYPE_NAME). Keywords ignore case.

`{ name = "condition" … }`

Named conditions in the `Condition::from_str` syntax. Quotes: `' " ''' """` (no escapes); triple quotes span lines.

`where { … }`

Condition names with AND / OR / NOT (or `&& || ! ~`) and parentheses.

`with meta { key = value … }`

Optional data for your own code: owner, severity, tags. It never affects matching.

Rules: one rule

Code

```
let text = fs::read_to_string("review.rule").unwrap();

match Expression::from_rule_str::<Student>(&text) {
    Ok(expr) => {
        let student = Student {…};
        println!("matches: {}", expr.matches(&student));
    }
    Err(e) => eprintln!("invalid rule: {e:?}"),
}

// same result as the ExpressionBuilder version:
// ExpressionBuilder::<Student>::new().add(…)…build(…)
```

What can go wrong

Stage	Error
Syntax	`ExpectingRuleKeyword`
	`ExpectingColon`
	`UnterminatedString`
	`ExpectingWhereKeyword`
Type	`TypeMismatch`
Conditions	`UnknownAttribute`
	`DuplicateConditionName`
Where	`UnknownConditionName`
	`EmptyExpression`

Syntax errors, including `with meta` mistakes such as `DuplicateMetaKey`, carry their position in the text, so a message can point at the exact place.

Rules: how they are read

Rule text	RuleParser	RuleDefinition	Expression	Your code

`rules.wx`
`rule … where { … }`
`with meta { … }`
	`RuleParser::new(&text)`
	Reads one rule
	at a time
		`next_rule_definition()`
		`name()` `type_name()`
		`meta()`
			`expression::<T>()`
			T chosen from
			`type_name()`
				`matches(&obj)`
				`evaluate(&obj)`
				+ the meta you read

↻ repeat until **`Ok(None)`**

One rule only: `Expression::from_rule_str::<T>(&text)` does all of this in one call.

One at a time

A RuleDefinition borrows the parser. Compile it and copy what you need before reading the next rule.

Many types, one file

Each rule names its type. Match on type_name() and compile with the right T.

Errors

A syntax error ends the reading. A rule that does not compile fails alone; you decide whether to go on.

Rules: many rules

rules.wx

```
rule adults: Student { adult = "age >= 18" } where { adult }
rule cheap:  Book    { low = "price < 10" } where { low }
rule top:    Student {
    good = "grade.average >= 9"
    math = "grade.math >= 9"
} where { good && math } with meta { owner = teachers }
```

Code

```
let mut parser = RuleParser::new(&text);
while let Some(def) = parser.next_rule_definition()? {
    let name = def.name().to_string();
    match def.type_name() {
        "Student" => students.push((name, def.expression::<Student>()?)),
        "Book"    => books.push((name, def.expression::<Book>()?)),
        other     => eprintln!("{name}: unknown type {other}"),
    }
}
```

What you get

Situation	Result
Next rule	`Ok(Some(def))`
End of the text	`Ok(None)`
Syntax error	`Err`, then `Ok(None)`
Does not compile	`Err` from `expression`, go on
Wrong T (type check)	`Err(TypeMismatch)`
Duplicate rule name	both returned

One rule at a time: a `RuleDefinition` borrows the parser, so compile it and copy what you need before reading the next rule. Pick T from `type_name()`, so one file can hold rules for several types.

Rules: metadata

rules.wx

```
rule big_logs: File {
    log = "extension is log"
    big = "size > 1000"
} where { log AND big } with meta {
    severity = high
    priority = 3
    tags     = [disk, "high load"]
}
```

Code

```
let mut parser = RuleParser::new(&text);
while let Some(def) = parser.next_rule_definition()? {
    let meta = def.meta();
    let severity = meta.get("severity").and_then(|m| m.value()).unwrap_or("low");
    let priority: i32 = meta.get("priority")
        .and_then(|m| m.value())
        .and_then(|v| v.parse().ok())
        .unwrap_or(0);
    if let Some(tags) = meta.get("tags") {
        for i in 0..tags.values_count() {
            println!("tag: {}", tags.array_value(i).unwrap());
        }
    }
}
```

Meta values

`key = value`

A word, number, date, IP or quoted text. Quotes are removed: `ops = "ops"`.

`key = [a, "b c"]`

A list, commas between values; `[ ]` is empty. A single value is a list of one.

`get("key")` `get(0)`

By key or by position. Keys ignore case and appear once per rule (`DuplicateMetaKey`).

`value()` `array_value(i)`
`values_count()`

First value (None for `[ ]`), value i, and how many. Values are text: you convert them.

SECTION 5 OF 5

05 Benchmarks

Speed and memory of the search operations

Performance considerations

Performance checks

Operation	Data set	Time/1M queries (ms)	Memory	Algorithm
Is-one-of	1000	85 ms	31K	BinSearch
Starts-with-one-of	1000	160 ms	10K	TRIE
Ends-with-one-of	1000	166 ms	10K	REV-TRIE
Contains-one-of	1000	330 ms	55K	DFA/NFA
Glob-Match	1000	1156 ms	4.4M	DFA+REGEXP

- **Data set** = the amount of string that were used to check against (**e.g. name contains […1000 strings…]**)
- **Time/1M queries** = how much time is required to run 1 millions of queries (conditions) of a specific kind (e.g. 1 million queries with different word that check if every word is contains any word from a specific list)

Each test was executed 10 times, and the median was used for time considerations

whereexpr

Write WHERE expressions. Filter anything.

crates.io
crates.io/crates/whereexpr

GitHub
github.com/gdt050579/whereexpr

docs.rs docs.rs/whereexpr

MIT License · Rust 2021 Edition